

Método XAI basado en agrupamiento para la explicación de errores en clasificación

Aurora Ramírez

Dpto. de Informática y Análisis Numérico
Universidad de Córdoba, Córdoba 14071
Email: aramirez@uco.es

Sebastián Ventura y José Raúl Romero

Dpto. de Informática y Análisis Numérico,
Instituto Interuniversitario de Investigación en
Data Science and Computational Intelligence (DaCSI)
Universidad de Córdoba, Córdoba 14071
Email: {sventura,jrromero}@uco.es

Resumen—Los modelos predictivos obtenidos por medio de aprendizaje automático están alcanzando altas cotas de precisión, pero no son completamente infalibles. Los errores de clasificación pueden tener gran repercusión en ciertas aplicaciones, generando desconfianza si no es posible entender por qué suceden. La inteligencia artificial explicable (XAI), cuyos métodos están dirigidos a explicar el comportamiento de los modelos predictivos y sus decisiones, puede ayudar a comprender las características de las instancias mal clasificadas. Este trabajo propone un método XAI basado en un algoritmo de agrupamiento para identificar los diferentes tipos de errores que comete cualquier algoritmo de clasificación binaria. A continuación, se utilizan métodos XAI locales sobre las instancias más representativas para analizar qué características son las más influyentes. Los experimentos con diferentes clasificadores y conjuntos de datos ilustran cómo el método propuesto es capaz de encontrar diferentes grupos de instancias que inducen a errores, permitiendo al usuario detectar situaciones en las que las predicciones deberían ser revisadas.

Index Terms—Inteligencia artificial explicable, clasificación binaria, errores, agrupamiento

I. INTRODUCCIÓN

El auge de los sistemas basados en inteligencia artificial ha propiciado la necesidad de analizar cómo funcionan sus métodos y qué implicaciones tienen sus resultados. Muchos modelos de decisión contruidos automáticamente son opacos o difíciles de entender, lo cual genera desconfianza en sus predicciones. En este contexto surge XAI (*eXplainable Artificial Intelligence*), que promulga la construcción de modelos capaces de explicar su proceso de decisión y las razones que subyacen a sus predicciones con el objetivo de favorecer la transparencia, aumentar la credibilidad y reducir sesgos [1].

Una de las causas que genera falta de confianza es el hecho de que las predicciones obtenidas por medio de aprendizaje automático (ML, *Machine Learning*) no son infalibles [2]. En contextos como la medicina, una predicción errónea puede tener graves consecuencias, por lo que es importante entender cuándo y por qué un modelo predictivo comete errores. XAI puede aportar una visión novedosa, identificando si dichos errores se deben a características concretas de los datos o son propiciados por el propio proceso de aprendizaje.

Los métodos XAI pueden orientarse a explicar el *modelo* predictivo o sus *salidas* [3]. En el primer caso, lo habitual es

que el método XAI tenga alcance *global*, frente a la explicación de salidas con carácter *local*. A su vez, el método XAI puede ser *específico* o *agnóstico*, en función de si depende o no del tipo de modelo predictivo. Finalmente, los métodos XAI pueden basarse en inducir, deducir o contrastar información obtenida de los modelos predictivos y/o sus predicciones. Un mecanismo inspirado en cómo los humanos razonan consiste en la generación de ejemplos y contraejemplos [4], pues permiten acotar la explicación a casos “prototipo”. Estudios con usuarios han demostrado que este tipo de explicaciones contribuyen a comprender el funcionamiento de ML [2].

A pesar de su relevancia y aplicabilidad, existen pocos métodos XAI orientados a explicar errores y los que hay son específicos del dominio de aplicación [5], focalizados en regresión [2] o que solo ofrecen información local [6]. En este artículo proponemos un método XAI agnóstico que combina técnicas de agrupación, para analizar los resultados del clasificador de forma global, con técnicas de explicación de instancias, para analizar de forma local instancias representativas entre aquellas mal clasificadas.

Las principales contribuciones de nuestro trabajo son: 1) el uso de un algoritmo de agrupamiento para la identificación de instancias prototipo, 2) una comparación entre métodos XAI locales para estudiar la influencia de las características en los errores prototipo identificados, y 3) un estudio experimental que analiza los errores de clasificación cometidos por diferentes algoritmos sobre varios conjuntos de datos. Nuestros resultados demuestran que el algoritmo de agrupamiento es eficiente y se ajusta al tipo de análisis requerido (cuando pocas instancias son erróneas). Además, los prototipos generados responden de manera diferente al análisis local, reflejando que no todos los errores cometidos por un algoritmo de clasificación tienen por qué responder a las mismas causas.

El resto del artículo se estructura como sigue. La sección II presenta trabajos relacionados. La sección III describe el método XAI propuesto. El marco experimental se detalla en la sección IV, seguido de los resultados y su discusión en la sección V. La sección VI recoge las conclusiones.

II. TRABAJO RELACIONADO

Nuestra propuesta está relacionada con trabajos que analizan las salidas de modelos de clasificación y, en especial, los

errores cometidos. Un primer trabajo aplica razonamiento simbólico sobre las salidas de un clasificador de imágenes con el objetivo de analizar las regiones mal identificadas [5]. Los objetos inferidos en la imagen representan la explicación, pero sin asociar aquellos mal identificados con el funcionamiento del clasificador. En el contexto de una aplicación real de ventas, se ha propuesto un método XAI local para explicar las instancias que presentan errores desmesurados durante la fase de prueba [2]. En este caso, se establece un umbral para filtrar las instancias, tras lo cual se estudia la correlación entre la predicción y cambios en las características.

Otros dos trabajos comparten ciertas características con nuestra propuesta. En primer lugar, Rizzi et al. aplican métodos XAI locales sobre cada una de las instancias mal clasificadas para así determinar las características que más influyen en las predicciones erróneas [6]. A continuación, se estudia la frecuencia con la que cada característica aparece en las instancias mal clasificadas. Tal y como explican los autores, combinar explicaciones locales para obtener explicaciones globales todavía requiere de un mayor estudio, aparte de que aproximar el comportamiento global a partir de métodos locales como LIME puede inducir a interpretaciones erróneas [7]. Por otra parte, Kim et al. han abordado las explicaciones basadas en ejemplos y contraejemplos [4]. Los autores plantean la necesidad de identificar partes del conjunto de datos que difieren de los prototipos seleccionados, utilizando para ello un algoritmo voraz. Su propuesta se basa en analizar la similitud entre instancias para encontrar las que difieren en mayor medida, aunque para ello utilizan todo el conjunto de datos. En sus experimentos comparan el algoritmo de extracción de prototipos con un algoritmo de agrupamiento (k-medoids).

III. DESCRIPCIÓN DEL MÉTODO

La Figura 1 muestra un esquema general del método XAI propuesto, el cual consta de tres fases: 1) entrenar y validar un clasificador, extrayendo las instancias que el clasificador predice erróneamente, 2) agrupar las instancias sobre las que el clasificador falla en su predicción, y 3) explicar las instancias prototipo sobre las que se han cometido los errores de clasificación. A continuación, se explica cada fase en detalle.

III-A. Entrenamiento y prueba

Esta fase engloba el procedimiento habitual para entrenar y probar un modelo de clasificación. Tras aplicar el preprocesado necesario (no representado en la figura), deben separarse los datos para entrenamiento (aplicando validación cruzada u otra estrategia) y prueba. El clasificador puede ser entrenado por medio de cualquier técnica (redes neuronales, máquinas de vector soporte, etc.), ya que el método propuesto es agnóstico.

Una vez construido el clasificador, se comprueba si el clasificador acierta o falla en su predicción sobre la partición de prueba. En caso de fallo, la instancia se identifica como falso positivo (FP) o como falso negativo (FN). Aunque aquí se ilustra el caso de un problema de clasificación binaria, el método puede ser fácilmente adaptado a problemas multi-clase separando los errores por clases.

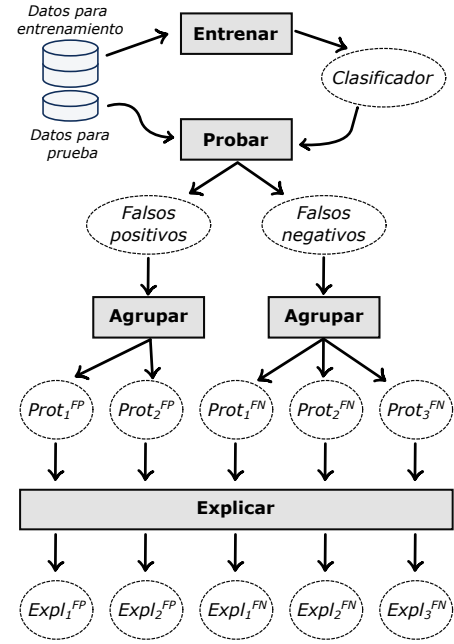


Figura 1: Visión general del método XAI.

III-B. Agrupación de instancias

En esta fase comienza el proceso de análisis de errores, el cual es equivalente entre FP y FN. El objetivo es encontrar grupos de instancias mal clasificadas para detectar las zonas de la distribución de los datos en las cuales el clasificador tiende a fallar. Un segundo objetivo es reducir la cantidad de instancias que se presentarán al usuario durante la fase de explicación, de forma que el análisis se centre en las más representativas. Además, los prototipos deben corresponderse con instancias del conjunto de datos para que le resulten realistas al usuario.

En base a los dos objetivos expuestos, en esta fase se emplea un algoritmo de agrupamiento llamado *Affinity Propagation* (AP) [8]. Este algoritmo posee dos características que lo hacen apropiado para los objetivos planteados: 1) no necesita preconfigurar el número de grupos a descubrir y 2) localiza, para cada grupo, la instancia más representativa a la que denomina *exemplar*. AP se basa en una estrategia de "paso de mensajes" entre las instancias para elegir cuál de ellas es el mejor *exemplar* para otras instancias, y determinar a qué *exemplar* debe asociarse cada una de las instancias restantes. El proceso se repite iterativamente hasta lograr grupos de instancias estables. Las reglas de actualización de los *exemplars* se basan en las siguientes medidas:

1. Similitud ($s(i, j)$), representada como una función de distancia entre dos instancias i y j . Por defecto, se considera la distancia euclídea.
2. Responsabilidad ($r(i, j)$), que indica el grado en que la instancia j es un buen *exemplar* para la instancia i considerando el resto de posibles *exemplars* para i .
3. Disponibilidad ($a(j)$), que calcula, para un *exemplar* j , el grado en que es representativo del resto de instancias.

AP tiene complejidad cuadrática respecto al número de instancias, por lo que puede ser costoso computacionalmente. No obstante, el subconjunto de instancias mal clasificadas será pequeño en comparación al total de instancias, por lo que el algoritmo es adecuado para nuestro propósito. Por otro lado, AP es sensible a varios parámetros, pudiendo no llegar a converger. Sus tres parámetros son: *damping*, un valor entre 0 y 1 que regula el paso de mensajes; *max_iter*, que fija el número máximo de iteraciones; y *convergence_iter*, que determina el máximo número de iteraciones antes de reajustar el número de grupos. Ante la posible falta de convergencia, nuestra experimentación incluye un análisis de la influencia de estos parámetros (Sección V-B). No obstante, la no convergencia de AP puede interpretarse como la existencia de un único grupo formado por todas las instancias mal clasificadas, eligiendo uno o varios *exemplars* de manera aleatoria [4].

Como resultado de esta fase se obtiene una separación de instancias FP y FN en k grupos (el valor de k no tiene por qué ser el mismo en ambos casos). Para cada grupo, se dispone además de una instancia *exemplar* que representa el prototipo sobre el cual se van a generar las explicaciones locales.

III-C. Explicación de errores

La última fase consiste en la generación de una explicación para cada uno de los prototipos de error identificados en el paso anterior. Puesto que cada prototipo es una instancia real del conjunto de datos, su explicación puede generarse por medio de un método XAI local para explicación de salidas. En concreto, se va a estudiar el funcionamiento de tres métodos:

- *LIME* [9]: este método construye un modelo lineal simple que se aproxima al modelo predictivo completo en el entorno de la instancia. Para ello genera instancias en su vecindad mediante un método basado en perturbación.
- *SHAP* [10]: es un método basado en la teoría de juegos que busca encontrar una distribución óptima de la importancia que tiene cada característica en la predicción devuelta por el modelo. Se trata de un método fundamentado en una base teórica que unifica otras propuestas.
- *BreakDown* [11]: este método también calcula la influencia de cada característica en la predicción obtenida. Su principal diferencia es que sumando la atribución que recibe cada característica se obtiene el valor de la predicción, lo cual facilita su interpretación.

IV. MARCO EXPERIMENTAL

Esta sección describe los experimentos que estudiarán el rendimiento del método propuesto. A continuación, se detallan los conjuntos de datos utilizados y la parametrización de los algoritmos.

El método XAI y la experimentación se han implementado en Python utilizando las siguientes librerías: *scikit-learn* [12], para el preprocesado de datos y la ejecución de los algoritmos de clasificación y agrupamiento; y *dalex* [13] para la ejecución de métodos XAI locales, el cual a su vez requiere el paquete *lime*¹ para la ejecución de dicho método. La experimentación

¹<https://github.com/marcotcr/lime> (accedido: 21/05/2021)

Tabla I: Conjuntos de datos y sus características

Nombre	Num. Características	Num. Instancias
Breast cancer	9	277
Column	6	310
Credit	15	653
Diabetes	8	768

Tabla II: Algoritmos y sus parámetros

Parámetro	Valores
Random Forest (RF)	
<i>n_estimators</i>	50, 100, 500, 1000
<i>max_depth</i>	10, 25, 50, ilimitado
<i>max_features</i>	$\sqrt{NC}$ (<i>auto</i>), todas
<i>min_samples_split</i>	2, 5, 10
<i>min_samples_leaf</i>	1, 2, 5
Support Vector Machine (SVM)	
<i>kernel</i>	lineal, radial, sigmoide
<i>gamma</i>	$1/(NC * var(d))$ (<i>scale</i>), $1/NC$ (<i>auto</i>)
<i>max_iter</i>	500, 1000, 5000, 10000, ilimitado
Affinity Propagation (AP)	
<i>damping</i>	0.5, 0.7, 0.9
<i>max_iter</i>	100, 200, 500
<i>convergence_iter</i>	5, 15, 30

se ha ejecutado en una máquina con Debian 8, ocho núcleos Intel Core i7-2600, CPU a 3.4 GHz y 16 GB de RAM.

IV-A. Diseño experimental

Como paso previo a la experimentación del método XAI, es necesario obtener instancias mal clasificadas para su análisis. Por ello, primero estudiaremos las salidas de dos algoritmos de clasificación bajo dos configuraciones de parámetros (por defecto y optimizados), con el objetivo de comprobar si afectan al número de errores cometidos. A continuación, se plantean dos experimentos para estudiar la influencia de los elementos que intervienen en cada fase del método XAI:

1. *Experimento #1*. El objetivo es estudiar el rendimiento del algoritmo de agrupamiento, con especial énfasis en la configuración de sus parámetros para garantizar la convergencia. Se considerarán los resultados sobre la salida del clasificador con parámetros por defecto y optimizados, a fin de establecer diferencias en cuanto al número de grupos identificados y sus características.
2. *Experimento #2*. El objetivo es comparar los métodos descritos en la sección III-C. Dada la limitación de espacio, se utiliza un único clasificador sobre un conjunto de datos según lo analizado en los experimentos previos.

IV-B. Conjuntos de datos

La Tabla I detalla las características de los conjuntos de datos seleccionados. Todos ellos tratan un problema de clasificación binaria, aunque varían en cuanto al dominio de aplicación, número de características (#C) y número de instancias (#I). Algunos de estos conjuntos de datos son habituales en experimentos con métodos XAI [14], [15].

Los conjuntos de datos han sido divididos en partición de entrenamiento (75 %) y de prueba (25 %). La partición de entrenamiento es la utilizada para el estudio de parámetros, aplicando a su vez validación cruzada estratificada con 10

Tabla III: Resultados de los clasificadores sobre la partición de validación

Conjunto de datos	RF_d				RF_p				SVM_d				SVM_p			
	Acc	F1	FP	FN	Acc	F1	FP	FN	Acc	F1	FP	FN	Acc	F1	FP	FN
Breast cancer	0.7571	0.5143	6	11	0.7500	0.4138	3	14	0.7000	0.1600	3	18	0.6857	0.4762	12	10
Column	0.7692	0.8421	13	5	0.7692	0.8393	12	6	0.8077	0.8649	13	5	0.7564	0.8224	12	6
Credit	0.8902	0.8784	9	9	0.8902	0.8816	11	7	0.8293	0.8250	20	8	0.8780	0.8734	15	5
Diabetes	0.7969	0.6829	14	25	0.7969	0.6829	14	25	0.7760	0.6504	16	27	0.7708	0.6393	16	28

particiones. La partición de prueba se utiliza para evaluar el clasificador resultante del estudio de parámetros (o con su configuración por defecto), y es de donde se extraen las instancias mal clasificadas para su posterior análisis.

IV-C. Algoritmos y métricas de rendimiento

Para la construcción de clasificadores se utilizan *random forest* (RF) y *support vector machine* (SVM). La Tabla II muestra los parámetros que se van a estudiar para cada algoritmo. El valor subrayado corresponde a la configuración por defecto del algoritmo en *scikit-learn*. Se han establecido valores tanto superiores como inferiores a los valores por defecto, salvo que estos correspondan al mínimo o máximo teórico. Para el caso de SVM, el parámetro *max_iter* no se utiliza en la configuración por defecto, pero se ha incluido en el estudio de parámetros para comprobar si se puede reducir el tiempo de ejecución sin que afecte a la precisión del algoritmo.

El proceso de optimización de parámetros consiste en combinar todos los valores de cada parámetro entre sí, procedimiento conocido como *grid search*. Para cada combinación de parámetros, se aplica la validación cruzada y se ejecutan 30 semillas aleatorias diferentes. De cada ejecución se extrae la matriz de confusión y las siguientes métricas de rendimiento: *accuracy*, *recall*, *precision* y *F1*. Aquella combinación con la media más alta en la métrica F1 se elige como la mejor.

Para la segunda fase, se van a comparar 27 combinaciones de los tres parámetros del algoritmo AP (ver Tabla II). Para cada una de ellas se realizan 30 ejecuciones con distintas semillas aleatorias. Los resultados se analizan en base a las siguientes métricas: número de grupos, *Silhouette Coefficient* (SC), *Calinski-Harabasz Index* (CHI) y *Davies-Bouldin Index* (DBI). Se han elegido estas tres métricas porque no requieren la existencia de clases etiquetadas con las que comparar la agrupación realizada. Es decir, se basan en analizar la dispersión de los grupos creados. SC y CHI son métricas a maximizar, mientras que DBI debe ser minimizada.

V. RESULTADOS Y DISCUSIÓN

Esta sección presenta los resultados de la experimentación, discutiendo a su vez los aspectos más relevantes del método.

V-A. Análisis de errores en clasificación

Como punto de partida del método XAI propuesto, es conveniente analizar cuántos errores cometen los clasificadores bajo diferentes configuraciones de parámetros, y de qué tipo (FP o FN) son dichos errores. Esta comprobación es apropiada en XAI, pues es habitual que su audiencia objetivo sean usuarios no expertos en ML que tienden a utilizar las configuraciones por defecto de los algoritmos.

La Tabla III recoge las métricas de rendimiento para cada algoritmo sobre la partición de prueba, tanto con la configuración por defecto (RF_d y SVM_d) como para la mejor combinación de parámetros (RF_p y SVM_p). Centrándonos en RF, ambas configuraciones coinciden en el número total de instancias mal clasificadas (FP+FN). Para dos conjuntos de datos (*Breast cancer* y *Column*), la configuración optimizada reduce el número de FP, pero a costa de aumentar el de FN en igual proporción. Para el caso de SVM, limitar el número de iteraciones conlleva una reducción en la fiabilidad del clasificador para los conjuntos *Column* y *Diabetes*. No obstante, para *Credit* se han reducido tanto los FP como los FN, mientras que para *Breast cancer* se mejora F1 gracias a un menor número de FN.

En general, se puede afirmar que la optimización de parámetros puede resultar ventajosa en ciertos conjuntos de datos, pero que la configuración por defecto tiene un rendimiento similar. En el contexto de nuestra experimentación, no realizar una optimización de parámetros no va a repercutir significativamente en el número de errores a analizar en las siguientes fases del método para los conjuntos de datos seleccionados.

V-B. Análisis del algoritmo de agrupamiento

Una vez identificado el subconjunto de FP y FN para cada algoritmo y conjunto de datos, podemos estudiar la influencia de los parámetros del algoritmo AP. Las tablas IV y V recogen el número de grupos (#G) y las medidas de evaluación de los grupos (SC, CHI y DBI) para las instancias mal clasificadas por RF y SVM, respectivamente. Los valores representan la media de las 27 configuraciones de AP (ver Tabla II), junto a la desviación estándar. Para cada algoritmo, se diferencia entre FP y FN, así como según su configuración de parámetros.

Un primer hecho a destacar es que AP no proporciona una división en grupos ($\#G = 1$) para las instancias FN de los tres conjuntos de datos con menos instancias. El número de FN varía entre 5 y 18 instancias, mientras que para *Diabetes* es igual o superior a 25. No obstante, AP sí propone una división en grupos para todos los casos de FP, aún cuando el número de FP es inferior al de FN. Esto parece indicar que existe mayor diversidad entre las instancias negativas, esto es, podrían haber sido clasificadas como positivas por diferentes motivos. Por el contrario, todas las instancias positivas clasificadas como negativas podrían responder a un mismo patrón. Además, para el caso de los FP, el número de grupos sí aumenta en función del número de instancias como cabría esperar.

Por otro lado, se aprecia una diferencia respecto al agrupamiento de FN para el conjunto de datos *Diabetes*. Para RF, los 25 FN se agrupan en 4-6 grupos, mientras que los 27-28 FN

Tabla IV: Resultados del algoritmo de agrupamiento para errores de RF

	Conjunto de datos	RF_d				RF_p			
		#G	SC ($\uparrow$)	CHI ($\uparrow$)	DBI ($\downarrow$)	#G	SC ($\uparrow$)	CHI ($\uparrow$)	DBI ($\downarrow$)
FP	B. Cancer	2.08±0.03	0.15±0.00	1.85±0.00	0.42±0.00	2.18±0.04	0.16±0.00	1.97±0.00	0.41±0.00
	Column	4.00±1.41	0.19±0.03	6.40±0.62	0.85±0.29	3.44±0.83	0.25±0.05	5.21±0.58	0.75±0.18
	Credit	3.67±0.47	0.18±0.13	3.21±1.06	0.81±0.56	3.78±0.42	0.25±0.11	3.76±0.99	0.72±0.58
	Diabetes	3.89±1.10	0.26±0.01	4.44±0.72	0.68±0.01	4.56±1.26	0.30±0.00	7.27±0.00	0.69±0.00
FN	B. Cancer	1.00±0.00	-	-	-	1.00±0.00	-	-	-
	Column	1.00±0.00	-	-	-	1.00±0.00	-	-	-
	Credit	1.00±0.00	-	-	-	1.00±0.00	-	-	-
	Diabetes	6.67±2.05	0.14±0.01	5.23±0.27	0.92±0.07	6.33±1.89	0.13±0.00	4.96±0.00	0.98±0.00

Tabla V: Resultados del algoritmo de agrupamiento para errores de SVM

	Conjunto de datos	SVM_d				SVM_p			
		#G	SC ($\uparrow$)	CHI ($\uparrow$)	DBI ($\downarrow$)	#G	SC ($\uparrow$)	CHI ($\uparrow$)	DBI ($\downarrow$)
FP	B. Cancer	2.14±0.05	0.05±0.00	1.23±0.00	0.52±0.00	3.22±0.63	0.18±0.01	3.20±0.18	1.08±0.10
	Column	3.78±0.42	0.29±0.12	8.18±2.59	0.52±0.41	3.78±0.42	0.29±0.12	8.18±2.59	0.52±0.41
	Credit	3.78±0.42	0.25±0.11	3.76±0.99	0.72±0.58	3.78±0.42	0.25±0.11	3.76±0.99	0.72±0.58
	Diabetes	4.56±1.26	0.30±0.00	8.30±0.00	0.70±0.00	4.56±1.26	0.30±0.00	8.30±0.00	0.70±0.00
FN	B. Cancer	1.00±0.00	-	-	-	1.00±0.00	-	-	-
	Column	1.00±0.00	-	-	-	1.00±0.00	-	-	-
	Credit	1.00±0.00	-	-	-	1.00±0.00	-	-	-
	Diabetes	4.73±2.73	0.18±0.00	5.92±0.00	1.16±0.03	4.73±2.73	0.18±0.00	5.92±0.00	1.16±0.03

 Tabla VI: Influencia de las características en los prototipos (FP del algoritmo RF_p para el conjunto de datos *Diabetes*)

Característica	LIME					SHAP					BreakDown				
	P_1	P_2	P_3	P_4	P_5	P_1	P_2	P_3	P_4	P_5	P_1	P_2	P_3	P_4	P_5
preg	-	-	+	-	-	+	-	+	-	-	≈ 0	-	-	-	-
plas	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+
pres	-	+	-	+	-	+	-	+	+	-	+	+	-	+	-
skin	-	-	+	-	+	-	-	+	-	+	+	+	+	-	+
insu	-	-	+	-	+	-	-	+	-	-	+	+	+	+	+
mass	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+
pedi	-	-	+	+	+	-	-	+	+	+	-	-	+	+	+
age	+	+	+	+	-	+	+	+	+	-	+	+	+	+	-

cometidos por SVM se han dividido en 2-4 grupos. Aunque el número de FN es parecido en ambos casos, cada clasificador se equivoca en instancias con características diferentes y que, por tanto, se agrupan de distinta manera. Los valores de desviación estándar indican una mayor variabilidad entre las distintas ejecuciones de AP para este conjunto de datos.

De entre las medidas de densidad, CHI es la que presenta una mayor variación entre conjuntos de datos con similar #G. En concreto, los grupos están menos cohesionados para el conjunto de datos *Credit* en comparación con *Column* y *Diabetes*, a pesar de que no existe una gran diferencia en #G. En general, los valores de desviación estándar indican que los resultados son muy estables entre las diferentes configuraciones de AP, incluso aunque #G varíe ligeramente.

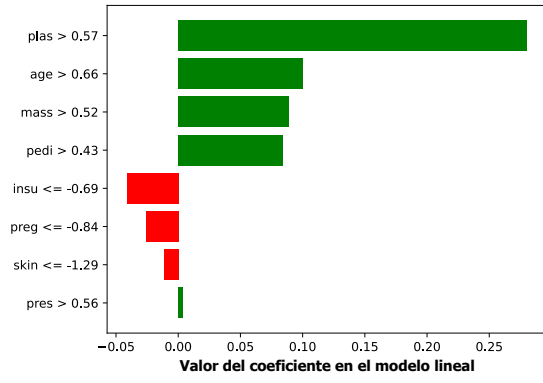
Respecto a los parámetros de AP, cabe señalar que algunas configuraciones no han logrado converger para los conjuntos de datos *Breast cancer* y *Diabetes*. Esto sucede especialmente cuando *damping* = 0.9 y *convergence_iter* = 5. En el resto de los casos, AP es bastante estable y su configuración por defecto es representativa del comportamiento general. El tiempo de ejecución ha sido siempre inferior a 0.25 segundos.

V-C. Análisis de las explicaciones locales

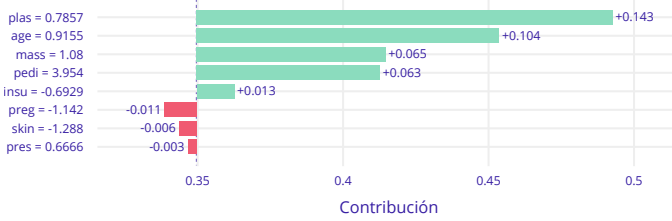
Para analizar las explicaciones asociadas a los prototipos, nos centramos en el algoritmo RF , por tener un comporta-

miento similar entre sus configuraciones, y el conjunto de datos *Diabetes*, para el que se produce un número relativamente alto de errores. Para la agrupación de errores, se ha ejecutado AP con parámetros por defecto. Como resultado, se obtienen cinco prototipos de FP y siete prototipos de FN. Por cuestiones de espacio, solo se realiza un análisis detallado de los FP. La probabilidad de pertenencia a la clase positiva devuelta por RF para cada FP es: $Prot_1^{FP} = 0.65$, $Prot_2^{FP} = 0.70$, $Prot_3^{FP} = 0.64$, $Prot_4^{FP} = 0.72$ y $Prot_5^{FP} = 0.56$. Estos valores ya nos indican que existen diferencias en cuanto al grado de error que comete RF en cada caso, pues no todos los valores están próximos a la barrera de decisión (0.5).

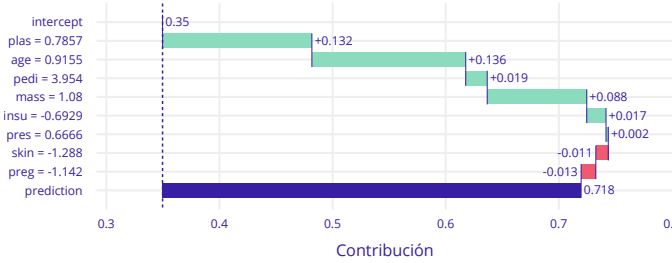
La Tabla VI muestra la influencia de las características en la predicción de cada prototipo (P), según *LIME*, *SHAP* y *BreakDown*. Dado que cada método calcula esta influencia de manera diferente, los valores concretos son poco representativos para la comparación y por tanto solo se indica el signo (+ si la característica contribuye positivamente a la predicción y - en caso contrario). Observamos que cada prototipo tiene asociada una combinación diferente de características positivas y negativas. Este hecho confirma que son instancias que responden de manera diferente a la predicción. Por tanto, podemos concluir que AP ha separado instancias que, de cara al usuario final, no van a proporcionar explicaciones repetidas. Por otro lado, los métodos no siempre coinciden en el tipo de



(a) LIME



(b) SHAP



(c) BreakDown

Figura 2: Explicación local del prototipo $Prot_4^{FP}$

atribución (+ o -) de una misma característica para un mismo prototipo. La forma en que cada método analiza el entorno de vecindad y los elementos del modelo de decisión podrían ser la causa.

A modo de ejemplo, la Figura 2 muestra la salida de cada método para $Prot_4^{FP}$. Es el caso en el que existe mayor coincidencia entre los métodos y, además, es el prototipo que obtuvo mayor probabilidad de pertenencia a la clase positiva. El orden en el que contribuye cada característica es bastante similar entre los tres métodos, siendo las características *plas* y *age* las más influyentes. En resumen, la inclusión de métodos locales es capaz de aportar más información al usuario acerca de las características que influyen en los errores cometidos. No obstante, es necesario estudiar sus propiedades en más detalle para entender sus divergencias y el impacto que puedan tener en la interpretación de los resultados por parte del usuario.

VI. CONCLUSIONES

En este trabajo se ha presentado un método XAI dirigido a explicar errores de clasificación en aprendizaje automático.

Tras identificar las instancias clasificadas erróneamente, el algoritmo de agrupamiento (AP) permite extraer un subconjunto representativo de errores, que son luego estudiados mediante métodos XAI locales. La experimentación realizada muestra la eficacia del algoritmo AP para revelar diferencias entre los errores cometidos, mientras que la elección del método local puede resultar más condicionante de cara a interpretar las causas asociadas a cada prototipo de error encontrado.

Como trabajo futuro nos planteamos extender el estudio experimental para poder extraer conclusiones más generalizables. También es necesario estudiar en más detalle el comportamiento del algoritmo AP respecto a los FN, así como las diferencias observadas entre los métodos XAI locales.

REFERENCIAS

- [1] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Information Fusion*, vol. 58, pp. 82–115, 2020.
- [2] A. Lucic, H. Haned, and M. de Rijke, "Why Does My Model Fail? Contrastive Local Explanations for Retail Forecasting," in *Proc. Conference on Fairness, Accountability, and Transparency (FAT*)*, 2020, pp. 90–98.
- [3] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A Survey of Methods for Explaining Black Box Models," *ACM Computing Surveys*, vol. 51, no. 5, pp. 1–42, 2018.
- [4] B. Kim, R. Khanna, and O. Koyejo, "Examples Are Not Enough, Learn to Criticize! Criticism for Interpretability," in *Proc. 30th International Conference on Neural Information Processing Systems (NIPS)*, 2016, p. 2288–2296.
- [5] M. Alirezaie, M. Långkvist, M. Sioutis, and A. Loutfi, "A Symbolic Approach for Explaining Errors in Image Classification Tasks," in *IJCAI Workshop on Learning and Reasoning*, 2018.
- [6] W. Rizzi, C. Di Francescomarino, and F. M. Maggi, "Explainability in predictive process monitoring: When understanding helps improving," in *Proc. International Conference on Business Process Management (BPM)*, 2020, pp. 141–158.
- [7] D. Garreau and U. von Luxburg, "Explaining the Explainer: A First Theoretical Analysis of LIME," in *Proc. 23rd International Conference on Artificial Intelligence and Statistics*, 2020, pp. 1287–1296.
- [8] B. J. Frey and D. Dueck, "Clustering by Passing Messages Between Data Points," *Science*, vol. 315, no. 5814, pp. 972–976, 2007.
- [9] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why Should I Trust You?": Explaining the Predictions of Any Classifier," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1135–1144.
- [10] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Proc. 31st International Conference on Neural Information Processing Systems (NIPS)*, 2017, pp. 4768–4777.
- [11] M. Staniak and P. Biecek, "Explanations of Model Predictions with live and breakDown Packages," *The R Journal*, vol. 10, no. 2, pp. 395–409, 2018.
- [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [13] H. Baniecki, W. Kretowicz, P. Piatyszek, J. Wisniewski, and P. Biecek, "dalex: Responsible Machine Learning with Interactive Explainability and Fairness in Python," *arXiv:2012.14406*, 2020.
- [14] B. P. Evans, B. Xue, and M. Zhang, "What's inside the Black-Box? A Genetic Programming Method for Interpreting Complex Machine Learning Models," in *Proc. Genetic and Evolutionary Computation Conference (GECCO)*, 2019, p. 1012–1020.
- [15] A. White and A. d'Ávila Garcez, "Measurable Counterfactual Local Explanations for Any Classifier," in *Proc. 24th European Conference on Artificial Intelligence (ECAI)*, 2020, pp. 1–7.